

Serie · Álgebra Lineal con Python

Sistemas lineales con Python

Resolver $Ax = b$ en una línea de NumPy



La idea

Un sistema de ecuaciones lineales se guarda como una matriz A y un vector b . Resolverlo es hallar el x que cumple:

$$Ax = b$$

Con NumPy, A y b son arrays. Tomemos este sistema:

$$x_1 + x_2 + x_3 = 3$$

$$x_1 - x_2 + 2x_3 = 2$$

$$x_2 + x_3 = 2$$



Montar el sistema

Cada fila de A es una ecuación; b son los términos de la derecha:



python

```
import numpy as np

A = np.array([[1, 1, 1],
              [1, -1, 2],
              [0, 1, 1]])
b = np.array([3, 2, 2])
```



Resolverlo

Una sola línea encuentra la solución:



python

```
x = np.linalg.solve(A, b)
```

Out

```
array([1., 1., 1.])
```

Obs. Por dentro, solve no invierte A : factoriza $A = LU$ (LAPACK) y despeja. Es $O(n^3)$, rápido y estable.



Verificar

Nunca confíes a ciegas: sustituye x y comprueba que Ax reproduce b . El operador @ es el producto matriz-vector:



python

```
np.allclose(A @ x, b)
```

Out

True



¿Y si no hay solución única?

solve solo funciona si A es invertible. Con una fila combinación de otras (aquí $\text{fila}_3 = \text{fila}_1 + \text{fila}_2$),

A es singular y falla:



python

```
C = np.array([[1, 1, 1],
              [1, -1, 2],
              [2, 0, 3]])
np.linalg.solve(C, np.array([3, 2, 1]))
```

Out

LinAlgError: Singular matrix

Quien decide es el rango: si $\text{rank}(A) < n$, hay 0 o ∞ soluciones, nunca una sola.



python

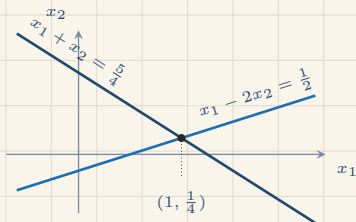
```
np.linalg.matrix_rank(C)    # -> 2  (< 3)
```

Obs. Para esos casos, lstsq da la solución de mínimos cuadrados aunque solve se rinda.



Verlo en 2D

Con dos variables, cada ecuación es una recta; solve devuelve el punto donde se cruzan:



Obs. Con tres variables cada ecuación es un plano, y la solución es donde los tres se cortan.

Regla de oro

Para resolver $Ax = b$ usa solve, nunca $\text{inv}(A)$

@ b: calcular la inversa es más lento, menos preciso y casi nunca hace falta.

Obs. NumPy apoya todo esto en LAPACK, la misma biblioteca de álgebra lineal que mueve el cómputo científico desde los años 90.



¿Resuelves tus sistemas a mano o dejas que NumPy piense?

Arranca la serie [Álgebra Lineal con Python.](#)

[Guarda](#) esto y [sígueme](#) para las siguientes en asanchezyali.com

